Solution Requirements

Date	04 July 2026
Team ID	
Project Name	Rising Waters - ML-Based Flood Prediction System
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

The requirements below define exactly what Rising Waters will do and how well it will perform. Functional Requirements (FR) describe specific behaviors and features of the system (what it should do), while Non-Functional Requirements (NFR) describe the quality criteria the system must meet (how it should perform).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	No user authentication is required – the prediction form is publicly accessible to any resident or official who wants a flood-risk estimate.	Low
2	Authorization levels	Single public user role only; there are no admin dashboards or restricted functionality in the current version of the application.	Low
3	External interfaces	No live third-party APIs are used; the app reads a local serialized model (model/floods.save) and historical Excel datasets used during training.	Medium
4	Transactions processing	No financial or multi-step transactions are involved; each prediction request is processed synchronously as a single form submission and response.	Medium
5	Reporting	The system generates a single result page (flood or no-flood) showing the predicted outcome and probability returned by the XGBoost model.	High
6	Business rules, etc.	Input feature values must be provided in the exact order and names defined in features.json before being passed to the scaler and model.	High
7	Compliance to laws or regulations	No personally identifiable information is collected or stored; only numeric meteorological values (temperature, humidity, rainfall, etc.) are processed.	Medium
8	Other	The application must run both via the local Flask development server and as a containerized service (Dockerfile, manifest.yml) for IBM Cloud deployment.	Medium

Step 2: Non-Functional Requirements (NFR)

S.N o	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The trained model is loaded once into memory, so each prediction request is processed with minimal latency.	Prediction returned in under 1 second per request.
2	Scalability	The app is containerized with Docker, allowing additional instances to be deployed as usage grows.	Handles at least 100 concurrent prediction requests without failure.
3	Security & Data Privacy	Only numeric weather values are submitted through the form; no personal data is collected or persisted.	No PII stored; all inputs processed in-memory only.
4	Reliability & Availability	The app runs behind a Gunicorn WSGI server in production, restarting workers on failure to keep the service available.	99% uptime per month.
5	Usability & Accessibility	The prediction form uses clear, descriptive labels for every input field so non-technical users can complete it easily.	A first-time user can submit a prediction in under 2 minutes.
6	Other	Maintainability: the trained model file can be retrained or replaced without any changes to the application code, since it is loaded dynamically via joblib.	Swapping the model file (floods.save) requires zero code changes.
